实验2 shell 初步

2.1 shell 基本操作

实验目的

使学生熟悉 shell 的基本操作，并使学生有机会使用 Linux 中最常用的数据处理命令。

实验内容与要求

独立完成实验内容，对实验步骤进行截图，并认真撰写实验报告。

要求

- 使用通配符来进行文件名的查找。
- 重定向标准输入、标准输出、标准出错。
- 使用管道操作，将一个进程的输出作为另一个进程的输入。
- 使用命令行续行符和命令组合。
- 在指定的目录中，对满足条件的文件名进行递归查找。
- 在文本文件中，查找符合指定模式的文本行。
- 将文本文件中的各行，按顺序排列。
- 显示一个文本文件的前几行或者后几行。
- 使用历史（`history`）命令。

内容

一、通配符

1. 使用 `cd` 命令，回到用户的主目录中。
2. 调用简单的 `ls` 命令，来列出用户的主目录中所有非隐藏文件。现在，使用 `ls` 命令与通配符 `*` 的组合来重新列出文件。这两个命令的输出有什么不同吗？为什么？
3. 进入 */usr/bin* 目录，仅列出以字母 a 开头的所有文件名：
   - `ls a*`
4. 列出所有只有两个字符的文件名：
   - `ls ??`
5. 列出所有以 a,b,c,d 四个字符之一开头的文件名：
   - `ls [a-d]*`
6. 列出所有不是以c-t开头的文件名：
   - `ls [^c-t]*`。
7. 上一条命令将有一个很长的输出。可以使用管道操作，将该命令的输出作为 `more` 或 `less` 命令的输入。
8. 你是否发现：你不想列出的文件名（如 R，S 开头的文件名）也列出来了？怎么办？

- `ls [^c-tC-T]*`

9. 回到你的用户主目录。

二、重定向

1. 使用 `cat` 命令与重定向，创建一个名为 *junk* 的文件。
   - 先随便输入几行内容，然后在最后一个新行的开始处，按 `<Ctrl-d>` 返回到 shell 提示符。
2. 使用 `cat` 命令在你刚刚创建的文件的末尾新加入几行文本。
3. 打印当前登录用户，并把显示情况重定向到一个文件中。

三、管道、过滤、`tee`、`&`

1. 使用 `ls` 命令，列出当前目录中的文件。记下文件的数量。
2. 使用 `ls` 命令，列出当前目录中的文件，并把输出重定向到文件 *temp* 中。
3. 使用一个命令来计算 *temp* 中的单词的数量。这个数量与你已经记下的数量相等吗？如果不相等，为什么？查看temp中的内容，而后删除它。
4. 现在使用一个管道操作来计算当前目录中的文件数目。这次所得到的结果与你已经记下的数量相等吗?
5. 调出你刚刚执行的命令，在管道中加入 `tee` 命令，将列表的输出存到文件 *junk2* 中。你得到了显示在屏幕上的文件数目了吗？查看 *junk2* 文件的内容，其中包含了你想要的数字吗?
6. 逆序列出当前目录中的文件。把结果输出到文件 *junk3* 中。
7. 计算这个逆序列表的单词数，并将结果输出到文件 *junk3* 的末尾。注意，重定向操作，有追加内容到文件末尾的功能。
8. 给出一个后台运行程序的例子，例如：`(sleep 20; echo hi)&`，再用 `ps` 命令打印当前的活跃进程，并给出当前活跃进程的父子关系分析。

四、命令序列以及续行符

1. 在同一行命令中，调用如下命令：显示日期；显示已经登录到系统中的所有用户名；显示当前目录的名称；显示当前目录中的所有文件名。这些命令，相互之间有联系吗?
2. 当一个命令太长，命令行一行无法容纳时，可使用命令行接续符，使命令能够完整地执行。你可以在你认为合适的地方，将一行结束。当你将命令输入完成时，shell 会马上运行它。

```
echo first \
second \
third
```

3. 这是一个使用了管道和重定向功能的超长的命令。
   - `ls -al | tee listing | wc -w | tee file1 file2 file3; wc -l file3 > file4`
   - 长列表当前目录中的所有文件，包括隐藏文件将上个命令的输出捕获（`tee`）到 *listing* 文件中，并且将这个输出送到下一命令中，以计算单词的数量。计算出单词的数量，并且该数量值同时按顺序存放到（`tee`）文件 *file1*，*file2*，*file3*中。最后计算出文件 *file3* 中的行数，并将该行数的值重定向到文件 *file4* 中。

五、find命令

1. 找到并显示出 */tmp* 目录中所有的文件名：
   - `find /tmp`
2. 在 */usr/bin* 目录中查找所有以字母 s 开头的文件名。而后，对它们自动执行 `ls -l` 命令。
   - `find /usr/bin -name "s*" | xargs ls -l`
3. 重复上一步的查找。不同的是，对查找的文件名执行 `ls -l` 命令时，采用与用户进行交互的方式。
   - `find /usr/bin -name "s*" -ok ls -l {} \;`
4. 在 */usr* 目录中，查找用户 root 的所有文件。而后，计算出这些文件的个数。对于该目录中的一些子目录，你是没有权限读取的，执行命令时将会产生一些错误信息。请将所有的错误信息重定向到一个名为 *errfile* 的文件中。
   - `find /usr -name "*" -user root 2> errfile | wc -l`
5. 显示 *errfile* 文件的内容。查看在执行上述的命令时，是否有错误发生。
6. 进入你自己的主目录中，执行以下命令：
   - 确认你正处在自己的用户主目录中。
   - 创建一个名为 *level1* 的目录。
   - 在子目录 *level1* 中创建一个名为 *letter1* 的零长度文件。
   - 进入到 *level1* 目录中。
   - 创建一个名为 *level2* 的目录。
   - 在子目录 *level2* 中创建一个名为 *letter2* 的零长度文件。
   - 进入到你的用户主目录中。
   - 在你的用户主目录中，列出所有以字母 l 开头的文件与目录（`ls -R l*`）。记录所显示的文件名与目录名。
   - 在你的用户主目录中，启动一个查找命令。查找出所有以字母 l 开头的文件与目录。记录找到的文件与目录。

     ```
     find $HOME -name "l*" -type f
     find $HOME -name "l*" -type d
     find $HOME -name "l*"
     ```

六、grep命令

1. 在 */etc/passwd* 中，找出所有以 sshd 开头的用户名所在的文本行。
   - `grep -e "^sshd" /etc/passwd`
2. 在 */etc/passwd* 中，找出所有以 t 开头的文本行。
   - `grep -e "^t" /etc/passwd`
3. 在 */etc/passwd* 中，找出所有包含数字 0-9 的文本行
   - `grep -e "[0-9]" /etc/passwd`
   - `grep -e "[[:digit:]]" /etc/passwd`
4. 重复上面的查找，但这次只显示所找到的文本行的数量。
5. 联合使用 `ps` 和 `grep` 命令，显示出系统中所有其它用户的进程。

- `ps -ef | grep -v "xxx"`

七、sort命令

1. 以字母顺序显示 */etc/passwd* 的内容。再以字母逆序显示其内容。
   - `sort /etc/passwd`
   - `sort -r /etc/passwd`
2. 以第三列的数字顺序排序，显示 */etc/passwd* 内容。
   - `sort -t : -k3n`

八、`head` 命令与 `tail` 命令

1. 显示 */etc/passwd* 文件的前10行的内容。
2. 显示 */etc/passwd* 文件的前5行的内容。
3. 显示 */etc/passwd* 文件的后10行的内容。
4. `tail` 命令也可以用于去除一些命令输出的头部信息。首先用一条命令列出在系统中运行的所有进程。注意该命令输出的标题信息。然后，再使用另一条命令列出在系统中运行的所有进程（不包括头部信息）。
   - `ps -ef | tail -n +2`
5. 可以通过修改 */etc/passwd* 文件的内容，重新设置用户的默认登录目录，如果要把 root 用户的默认登录目录改为 */home/root*，应该如何操作?

九、history命令

1. 列出最近输入的20条指令：
   - `history 20`
2. 使用命令记录号执行一条历史命令：
   - `!2171`
3. 重复执行一次命令：
   - `!!` 或 `up` 键
4. 执行最近一次以 `ls` 开头的指令：
   - `fc ls` 或 `!ls`
5. 在 `history` 命令文件中查找 `mkdir` 命令：
   - `cat .bash_history | grep mkdir`
   - `history | grep "mkdir"`

2.2 系统配置文件及环境变量

实验目的

熟悉 Linux 中常用的系统（用户）配置文件以及几个重要的环境变量。

实验内容与要求

独立完成实验内容，对实验步骤进行截图，并认真撰写实验报告。

要求

- 了解常用系统配置文件。
- 了解常用用户配置文件。
- 常用环境变量的设置。

内容

一、配置文件相关操作

1. 查看系统配置文件：使用 `vi` 命令分别打开 */etc/profile*, */etc/bash.bashrc*, *~/.profile*, *~/.bashrc*, *~/.bash_logout* 文件进行查看，分析 */etc/bash.bashrc*。
2. 修改配置文件，在当前用户登录时打印登录时间。
3. 修改配置文件，在当前用户登录时打印问候语：“hello，I am xxx, now I am working”。这里的xxx是该登录用户的用户名。
4. 思考：系统配置文件与用户配置文件的优先级关系，例如：在系统配置文件中设置 `$PATH` 变量，而在用户配置文件中又重新设置该变量，那么在该用户登录后，`$PATH` 变量是什么。

二、shell的环境变量

1. 查看当前系统的标准变量，分别说明它们的含义：
   - `$HOME`、`$IFS`、`$PATH`、`$PS1`、`$PS2`、`$SHELL`、`$TERM`、`$HISTSIZE`、`$HISTFILE`、`$HOSTNAME`、`$USER`、`$PWD`
2. 修改当前的提示符格式为：[登录用户@当前工作目录 登录时间]

   ```
   PS1=[\u@\w `date`]
   ```

3. 修改 `$HOME` 变量的值为 */tmp*，之后使用 `cd` 跳转到用户主目录。
4. 使用 `pwd` 查看当前路径。

2.3 shell编程

实验目的

使用户熟悉基本的 shell 程序设计方法，包括命令行参数检测、变量设置、文本建立和输入、程序运行控制和输入输出重定向等。

实验内容与要求

独立完成实验内容，对实验步骤进行截图，并认真撰写实验报告。

要求

编写一个 shell 程序，将用户从键盘输入的文本附加到一个指定的文件中。如果该文件不存在，则新建立该文件；如果该文件已经存在，则把键盘输入的文本附加到该文件后面。输入内容及行数不限定，以空行表示输入结束。运行结束时显示该文件内容。

内容

一、建立 shell 程序 exam1.sh

1. 用 `vi` 创建 shell 程序 exam1.sh。程序头应有几行以 `#` 开始的简单的程序功能说明行（需要设置 hashbang），其它主要语句前后都应有注释。
2. 通过检测变量 `$#` 的值判断命令行参数的个数是否正确，如正确则程序继续运行下去；否则提示用户该命令的运行格式（`Usage: exam1.sh filename`），并退出。
3. 设置变量 `$INPUT`，并赋予任意一个非空的字符串：`INPUT="x"`
4. 用 `echo` 语句在屏幕上提示用户输入任意一行文本（空行表示整个输入结束）。
5. 用 `while` 建立一个无限循环，循环退出条件为 `$INPUT` 为空，循环体为用 `read` 语句读入一行到变量 `$INPUT` 中，用 `echo` 命令将该变量的内容追加（`>>`）到用户在命令行上指定的文件名中。
6. 用 `cat` 命令将该文件的内容显示在屏幕上。
7. 将源代码粘贴在实验报告中.

二、用 `chmod` 命令设定 exam1.sh 的执行权限

1. 用 `ls` 命令查看刚建立好的 exam1.sh 的读写保护模式。
2. 用 `chmod` 命令修改 exam1.sh 的读写保护模式，使用户自己拥有执行权限。

三、运行程序

1. 在当前目录下直接用程序名 exam1.sh 运行该程序。如果出现类似提示：“exam1. sh: not found”，是什么原因？如何调整？
   - 用 ./ 执行 exam1.sh
   - 用 `bash` 执行 exam1.sh
   - 用 (.) 执行 exam1.sh
2. 试分析以上几种执行方法的区别。
3. 试试不带参数或参数个数大于等于两个来运行 exam1.sh 程序，看程序能否正常判断，并显示提示信息后退出。
4. 用一个新文件名 newfile 作为参数正常运行 exam1.sh，任意输入若干行，并以一个空行结束输入。程序能否显示前面的所有输入行后结束运行。
5. 用 `ls` 命令查看当前目录下是否有新建立的 newfile 文件，并用 `cat` 命令查看该文件的内容。
6. 再次用 newfile 作为参数运行 exam1.sh，运行结束后查看 newfile 文件中本次键盘输入的内容是附加还是覆盖文件中原有的内容。